

25 utility multipliers

How to make the engines worth several times what they are worth today — predictable, objective, consistent

The premise behind the arithmetic: **engines in isolation add; engines that feed each other multiply**. The site has roughly 125 shared engines and a mean page-wiring score of 3.3 out of 10. Every engine that gains a consumer gains a second reason to exist, and every engine that gains a *producer* stops being a form the user has to fill in. Most of the multiple below comes from connection, determinism and reuse — not from new features.

Each item states the change, the reasoning, the files, and how you would know it worked.

A. Determinism and reproducibility — the foundation everything else stands on

1. Make every projection a pure function of a versioned input bundle

Now: figures are computed from whatever the engine reads at call time — live series, today's constants, current code.

Change: wrap every projection in `run(inputs, assumptionSetId)` where `assumptionSetId` pins the constants and the as-of dates of every series used. Store the bundle with the result.

Why it multiplies: a plan you cannot reproduce is a plan you cannot defend, audit, compare, or re-run. This one change turns every output from a screenshot into a record. It is the precondition for items 2, 3, 4, 9, 18 and 22 — which is why it is first.

Files: `new shared/assumptionSets.ts`; every `shared/*Engine.ts` takes the id.

Check: re-running a stored plan with its bundle reproduces the figures byte for byte, asserted by a test.

2. Seed every stochastic engine and store the seed

Now: `monteCarloEngine` and `policySurvivalMonteCarlo` draw without a recorded seed.

Change: every random draw takes an explicit seed (the other session's `mulberry32` in `sequenceStress.ts` is already the right primitive); the seed goes in the result.

Why: "run it again and show me" currently produces a different answer, which reads as unreliability even when it is correct. Same seed, same distribution, same chart.

Files: `shared/monteCarloEngine.ts`, `shared/sequenceStress.ts` (once ported).

Check: two runs with the same seed are identical; with different seeds, they differ.

3. Freeze an "as-of" snapshot with every client plan

Change: when a plan is saved, snapshot the exact series values used (CPI print, PMMS rate, zip index, carrier rate) rather than a reference to the live series.

Why: six months later the advisor can say "here is what we knew on the day, here is what we know now, here is the difference" — which is a conversation no competitor can have, and it is entirely free once item 1 exists.

Files: `drizzle/schema.ts` `new plan_snapshots`; the save path in the planner router.

Check: a stored plan renders identically a year later.

4. Golden-file tests on every headline figure

Change: for the 31 featured pages, store the expected headline figure for a fixed input bundle; fail the build when it moves without a version bump.

Why: today an engine change can silently alter every client-facing number. This makes an intentional change visible and an accidental one impossible.

Files: `server/goldens/*.json`, one test.

Check: editing a rate constant fails the suite until the golden is updated deliberately.

B. Connection — where the actual multiple lives

5. One `Situation` object, produced once, consumed everywhere

Now: the fact finder, the genome and the planner each hold their own shape; the user retypes.

Change: `Situation` (in `sequencePlanner.ts`) becomes the household. Fact finder emits it, genome enriches it, strategy fit reads it, planner plans from it, provenance traces it, the brief prints it.

Why it is the single largest multiplier on the list: every engine currently costs the user a form. Remove the forms and every engine becomes free to reach. Five engines behind one form are worth far more than five engines behind five forms — the user completes one.

Files: `shared/sequencePlanner.ts` (extend), `shared/clientFactFinder.ts`, `shared/genomeStrategyFit.ts`.

Check: a household completes the fact finder and every one of the 52 rated pages renders populated with no further input.

6. Make every engine output link to the engine that consumes it next

Change: each result object carries `nextEngines: Array<{id, why, path}>`.

Why: turns 125 isolated tools into a graph a user can walk. The zip engine's appreciation figure should offer the planner; the planner's policy stage should offer the policy lab; the erosion trajectory should offer the Roth ladder.

Files: a `Handoff` type in `shared/` plus one field per engine result.

Check: the integration scorecard's `crossLinked` gap falls below 40.

7. Publish a single typed façade per domain, as done for the capital stack

Change: `realEstateCapitalStackEngine.ts` and `historicalMarketRegimeEngine.ts` (both written this session, tested, 17 tests) are the pattern. Add `taxEngine`, `insuranceEngine`, `householdEngine` façades.

Why: a caller currently needs to know five module names and their call order. Six façades make the whole system addressable by anyone — including an AI channel, a partner API and a future mobile app. It is the difference between a library and a platform.

Files: five new façade modules.

Check: any headline figure reachable in one call from a façade.

8. Wire the orphaned `russell-capital/` engines into the deployed app

Change: the ten engines in `master_build_correlation.md` §6.

Why: ~5,900 lines of finished, tested code that executes nowhere. This is the cheapest capability per hour available anywhere in the project — the work is already paid for.

Check: `grep -rl "russell-capital/" russell-capital-systems/` returns files instead of nothing.

9. A plan diff engine

Change: `diffPlans(a, b)` returning which stages changed, which assumption moved, and what it cost.

Why: the second-most-common advisor question after "what should I do" is "what changed". Answering it mechanically is a product other planners do not have.

Files: `new shared/planDiff.ts`.

Check: changing one input shows one changed stage and its dollar delta.

C. Objectivity — figures that can be checked

10. Adopt `UnsourcedFindingError` everywhere

Change: promote the other session's pattern into `shared/sourcing.ts`; every client-visible number routes through `assertSourced`.

Why: a build-time guard cannot catch a runtime value. This makes an unsourced figure structurally impossible rather than discouraged. It is the strongest single integrity upgrade available and the code already exists.

Check: a deliberately source-less value throws in a test.

11. A `FigureTrace` on every featured page

Now: three traces for 115 pages.

Why: provenance on one page is a curiosity; provenance on every page is a category difference. It is the thing that lets an advisor hand a figure to a sceptical accountant.

Files: `shared/provenance.ts`.

Check: provenance gap on the scorecard reaches zero for the featured 31.

12. Sample size travels with every base rate, enforced by type

Change: the regime façade already does this (`ConditionalOdds` carries `n` and `thin`). Make the `type` require it: a probability may not be rendered without its `n`.

Why: it is what separates a base rate from a guess. At a ten-year horizon the left bucket holds fourteen windows and the right holds five — so the honest output is "thin", and the engine returns `null` for the swing rather than a number. That refusal is worth more than a confident figure.

Check: `powerSwing` returns null when either bucket is thin — already asserted.

13. A calibration ledger

Change: record every prediction with its date and inputs; compare against what happened; publish the record as a page.

Why: nobody in this industry publishes their hit rate. Doing it converts "trust us" into "check us" and is defensible precisely because it will sometimes look bad.

Files: new `calibration_log` table, a page, a monthly job.

Check: the page shows at least one resolved prediction with its error.

14. Freshness as a first-class state

Change: every dated field renders its age; anything over 90 days shows a flag; the scorecard counts stale fields.

Why: the difference between "sourced" and "currently true". A rate from March presented without its date is worse than no rate.

Check: a fixture dated 100 days ago renders the flag.

15. Show the refusal, not just the result

Change: the planner already returns a reason for every refused move; surface refusals on every engine — why a strategy is blocked, why a lender is unavailable, why a figure is missing.

Why: an empty result teaches nothing; a refusal with a clause teaches the user what to change. It also demonstrates the system knows the rules, which is the actual product.

Check: every engine's result type has a `refusals` field with reasons over 20 characters.

D. Legal accuracy — wording that is both safer and truer

These are accuracy corrections. Each replaces a phrase that implies a promise with one that states what is actually known. They reduce exposure because they are *more correct*, not because they hide anything.

16. A single vocabulary module, enforced by test

Change: `shared/complianceVocabulary.ts` holding forbidden→preferred pairs; a test scans every page, PDF template and AI channel output.

Why: the site already has this pattern working for one phrase — `patentStatus.ts` forbids "patent pending" while nothing is filed, and `patentClaimGuard.test.ts` scans every surface. Generalise the mechanism that already works.

Replace	With	Reason
"guaranteed returns"	"contractually guaranteed minimum, as stated in the policy form"	The guarantee is a contract term with conditions, not a return
"tax-free"	"not currently taxable as income under [cite]"	Tax-free is a conclusion; the citation is a fact
"you become your own bank"	"you borrow from the insurer against your cash value"	The site's own dossier already corrects this; make it a rule
"risk-free"	"no market risk to principal; other risks remain, including [list]"	Removes an absolute the product does not support
"will grow to"	"projects to, on the assumptions shown"	The assumption is the load-bearing part
"eliminates taxes"	"defers, reduces or changes the character of the tax, as shown"	Elimination is rare and specific
"safe money"	"principal-protected by contract, subject to carrier claims-paying ability"	Names the actual risk
"we recommend"	"on these inputs, the engine ranks X first because Y"	Recommendation may be a regulated act; a ranking with reasons is a statement of arithmetic
"infinite banking"	keep as the searched term; define correctly in the first sentence	Already done — the simulator never uses "infinite" and a test enforces it
"beat the market"	"the historical series returned X over Y, sourced"	Replaces a promise with a record
"patent-pending"	<code>statusSentence()</code>	Already enforced; 57 drafted, 0 filed

Check: the vocabulary test passes across pages, PDFs and channel output.

17. Every projection carries its assumption set visibly

Change: no chart or table renders without the rate, horizon and source panel beside it.

Why: an illustration without assumptions is the thing regulators object to most, and it is also the thing that makes the figure useless later.

18. Suitability capture before any strategy ranking

Change: record what the household stated about objective, horizon, liquidity need and risk capacity; render it on every ranked output.

Why: a ranking without a recorded suitability basis is exposed. With one, the ranking is evidence of process.

19. Separate "education", "illustration" and "advice" as page classes

Change: every page declares its class; the class drives the disclaimer, the retention rule and whether an advisor signature is required.

Why: one disclaimer for 115 different pages is either wrong for most of them or so broad it protects nothing.

Files: `shared/calculatorCatalog.ts` gains a `class` field; `shared/loginDisclaimers.ts` keys off it.

20. An immutable client-facing audit trail

Change: every figure shown to a named client, with its as-of and assumption set, appended to a write-once log.

Why: answers "what did you show them, and when" definitively. This is the single most useful artefact in any dispute, and item 1 makes it nearly free.

E. Reach — the same engines, more consumers

21. A read-only partner API over the façades

Change: expose the six façades as authenticated REST/JSON.

Why: every engine gains consumers outside the website — a CRM, a spreadsheet, a partner firm, a future mobile app. Same code, several times the use. This is the clearest literal multiplier on the list.

22. Deterministic PDF generation from a stored plan

Change: a plan renders to the same PDF forever, because the assumption set is pinned.

Why: the advisor's actual deliverable. Makes the site the source of the document rather than a step before Word.

23. Bulk mode for advisors

Change: run the planner across a book of clients; return who has the largest gap between current position and best legal plan.

Why: converts a per-client tool into a prospecting engine. One advisor with 200 clients gets 200 uses instead of one at a time.

24. A "what would have to be true" inverter

Change: for any stage, compute the input value at which it stops paying — the 1.34 renovation uplift break-even generalised to every gate.

Why: the most useful sentence a planner can produce is "this works if $X \geq Y$, and here is your X ". It converts a projection into a decision rule, and it works on every engine that has a threshold.

Files: new `shared/breakEven.ts`; it already exists in miniature inside the cycle engine.

25. Wire every engine into the memory bank, and prove it

Change: every catalogue engine belongs to a memory group; `unwiredGroups()` returns empty; the scorecard's brain gap reaches zero.

Why: an engine the twelve channels cannot use is an engine the client must find alone. The brain is the distribution channel for all 125 engines at once — the highest-leverage single wire in the system, which is why it carries weight

1.5 on the scorecard.

Check: ask a channel a question that requires an engine added this week; it answers with the figure and names the engine.

The arithmetic, honestly

I cannot promise a specific multiple, and a number like "800%" is a goal rather than a forecast. What I can say precisely:

- **Item 5 alone** (one situation) changes the number of engines an average household actually reaches from roughly one to potentially all 52 rated pages, because the cost of reaching the next one falls to zero.
- **Item 8** adds ~5,900 lines of finished capability at the cost of wiring.
- **Item 21** multiplies consumers per engine rather than engines.
- **Items 1–4 and 10–15** do not add capability at all; they make existing capability *trustworthy*, which is what converts a demo into a product.

Do 5, 10, 8 and 1 first, in that order. They are the four the rest depend on.